ARCHITECTURE & ENGINEERING PLAN

Technical Approach Report

Evidence-to-Decision Intelligence for a verified, human-reviewed resource planning prototype

TECH-01

AAKAASH ADHINAYAK

VAYU NEXA AI | Alchemy Architects | SIH 2026

Prepared 20 October 2026 | Design assessment, not a certification

1. Executive assessment

AAKAASH ADHINAYAK is positioned as an evidence-driven resource-planning and analytics platform for SIH 2026 problem statement SIH26250. The project identity and design direction are established in the supplied research conversation. The defensible technical proposition is the full traceable path from source record to validated decision: import, validate, normalise, scenario-build, optimise, independently verify, explain, replan, audit and export evidence.

Evidence note: The public URL resolves to a page titled "AAKAASH ADHINAYAK - Verified Intelligence for Adaptive Resource Planning", but the available research interface did not expose navigable page text, runtime interactions, source code, API traces or test results. Therefore this report distinguishes observed deployment metadata and the team-supplied blueprint from proposed design and unverified implementation. It is not a source-code audit, security assessment, or certification of the live system.

Recommended technical thesis: “Evidence-to-Decision Intelligence” - preserve provenance and freshness, solve a constrained generic planning problem, independently verify every candidate plan, and make the evidence and trade-offs explorable in both 2D and 3D.

2. Live deployment review: what is and is not verified

The submitted URL is publicly reachable in the web retrieval layer and is titled “AAKAASH ADHINAYAK - Verified Intelligence for Adaptive Resource Planning”. The same access path returns no extractable application body. Without a browser session capable of clicking through the running app, source bundle, repository access or runtime logs, it is not possible to assert that any specific page, button, AI tool, database write or chart currently works.

Surface	Current evidence	Verification needed
Public URL and identity	Observed title is consistent with the project identity.	Capture full-page screenshots and route list; record build/version.
Rendered UI and navigation	Not inspectable through the available text retrieval.	Browser walkthrough of every route, empty/error/loading states and mobile breakpoints.
Data connectors	Not independently verified.	Network traces, endpoint response, provenance metadata and failure-path tests.
Optimiser and validation	Not independently verified.	Source, deterministic fixtures, tests and independent validator results.
AI and database persistence	Not independently verified.	Server-side route review, configured secret handling, DB rules and live smoke tests.

3. Reference architecture

The architecture should separate presentation, domain rules, data-source adapters, solver, independent verification, AI tools and audit records. A React/TypeScript client is appropriate for the dense interactive UI. A typed server API should own integrations and protected work. Persistence may use Firebase Authentication and Firestore if configured and tested; otherwise the application must label local/demo storage accurately. Google AI Studio supports deployment of full-stack applications through Cloud Run, but deployment support does not prove that a given feature or integration works [R2].

Layer	Responsibility	Evidence to add
Web client: React + TypeScript	Accessible pages, forms, tables, charts, 2D/3D views and responsive layouts.	Route tests, keyboard test, mobile screenshots, console-clean walkthrough.
Typed API/server	Auth, input schemas, data-source proxies, solver calls, exports, error handling.	API contract tests, auth tests, secret scan and structured error responses.
Canonical domain model	Source, resource, task, inventory, event, scenario, run, validation and forecast records.	Schema validation, stable IDs, versioning, migration tests.
Optimisation service	Deterministic constraint formulation, objective weights and recorded status/runtime.	Feasible and infeasible fixtures, timeout and invalid-model tests.
Independent validator	Recomputes all applicable hard constraints without trusting solver status.	Mutated invalid schedule rejected; valid schedule accepted.

Layer	Responsibility	Acceptance evidence
AI assistant tools	Retrieval and comparison of actual records, evidence-linked explanations.	Grounding tests, unknown-data test, prompt injection test, no-action authority.
Evidence ledger	Input snapshot, source versions, solver/validator version, results and human review state.	Export reproduces displayed run and can be rechecked.

4. Data flow and domain contracts

Use explicit typed schemas and validate on the server, even if the frontend performs an early usability check. Preserve raw inputs and a normalised representation; never silently fill missing values. Unknown, stale and conflicting values are first-class states rather than nulls that happen to be ignored. Each important value should point to its origin, observation/retrieval time and transformation history.

- SOURCE RECORD: source ID, publisher, URI, retrieval time, observation time, format/schema version, licence note, content hash and availability state.
- RESOURCE: generic ID, category, availability interval, maintenance status, capacity, capability tags, site label and source snapshot.
- TASK: generic ID, priority, duration, time window, dependencies, required tags and status.
- SCENARIO: immutable scenario/version reference, selected source snapshots, objective weights and creation time.
- OPTIMISATION RUN: solver/version, configuration, status, objective, runtime, assignments and unassigned tasks.
- VALIDATION RESULT: validator version, applicable checks, pass/fail list, violations and final state.
- AUDIT EVENT: action, actor role, timestamp, reason, affected records and review outcome.

5. Solver design and independent verification

Represent generic task-to-resource assignment and time intervals in a constraint model. Hard constraints must never be softened to make the dashboard look successful. Candidate hard constraints include resource availability, no double booking, maintenance exclusion, capability match, capacity limits and dependency ordering. Weighted soft objectives may minimise delay and unassigned tasks, reduce needless changes, balance workload and improve generic utilisation. Keep hard constraints and preference weights visibly separate.

Google OR-Tools CP-SAT is a suitable candidate when the deployment can run the library. Its documented status set includes OPTIMAL, FEASIBLE, INFEASIBLE, MODEL_INVALID and UNKNOWN [R6]. Map solver states carefully and do not translate UNKNOWN into failure or success. Record time limit, runtime and model configuration. A separate validator should recompute constraints from the returned schedule. Mark a schedule VALIDATED only when that validator has actually run and all applicable hard checks pass.

State	Meaning in the prototype
VALIDATED	Independent validator ran and every applicable hard constraint passed.
REVIEW_REQUIRED	Plan exists but required evidence or a human review remains outstanding.
INFEASIBLE	Solver determined no feasible assignment within the model and constraints.
TIMEOUT / UNKNOWN	Search did not establish a definitive outcome within the configured conditions.
MODEL_INVALID / REJECTED	Input/model invalid or candidate failed independent validation.

6. Public data and dataset strategy

Start with one external integration that can be fully implemented and tested. AviationWeather.gov offers public METAR observations, TAF forecasts, other products and documented query constraints; it also documents no CORS support, so a server-side connector is the appropriate design [R3]. Display source, endpoint, retrieval time, observation time, timezone where known, freshness and errors. Treat weather as contextual public data, not a safety clearance or defence intelligence.

For predictive maintenance, investigate an obtainable dataset from NASA PCoE. The repository includes prognostics datasets such as bearings, batteries and turbofan degradation simulation; the latter is a simulation-derived dataset, not real Indian military engine telemetry [R5]. Before selecting a dataset, download the exact archive, inspect the README, document the target and licence, and create reproducible train/validation/test splits. If data is not available, display “Experiment not yet trained” rather than invented metrics.

Source class	Allowed use	Required label
Public live	Only if endpoint fetch actually succeeds; cache with timestamps and source metadata.	PUBLIC-LIVE + retrieval/observation time
Public historical	Retrospective analysis and model experiments.	PUBLIC-HISTORICAL + dataset version
Team-uploaded	Only after source, permission, schema and privacy review.	TEAM-UPLOADED + reviewer decision
Synthetic test	Solver, failure-path, scale and controlled-replanning demonstrations.	SYNTHETIC-DEMO prominently shown
Unavailable/unknown	Block dependent decision or request human review.	UNAVAILABLE / UNKNOWN

7. VAYU NEXA AI and human oversight

The assistant should be a read-only, evidence-grounded copilot initially. Provide typed tools that retrieve current scenario inputs, data-quality issues, solver results, validator results, public-source metadata, audit events and run comparisons. Answers should cite record IDs and source timestamps and separate verified facts, model predictions, assumptions and unknowns. The language model must not approve a plan, make an unverified API claim, fabricate citations, or mutate operational planning state. If Gemini is unavailable, a clearly labelled fallback should explain that generative AI is not configured. Server-side secret storage and request logging discipline are mandatory.

Adopt risk assessment and monitoring consistent with the NIST AI RMF, which emphasises validity/reliability, safety, security/resilience, accountability/transparency, explainability, privacy and managed harmful bias [R7]. A language-model answer is explanatory, not a substitute for deterministic calculations or human authority.

8. Security, accessibility and reliability

- Role-based access for Viewer, Analyst, Planner, Reviewer and Administrator; enforce access on the server/API, not merely by hiding UI elements.
- If Firestore is used, write least-privilege Security Rules and test them; privileged server libraries also need correctly scoped IAM because they do not rely on client rules [R10].
- Review API endpoints for broken object-level authorisation, broken authentication, excessive property exposure, unrestricted resource use and unsafe third-party integrations against OWASP API Security Top 10 [R9].
- Apply secure development practices and release checks informed by NIST SSDF [R8]: dependency scans, secret scans, review, tests, error handling and reproducible release notes.
- Use WCAG 2.2 as an accessibility baseline: keyboard operation, visible focus, non-colour-only status cues, reduced motion, accessible chart alternatives, responsive layout and readable contrast [R11].
- Implement explicit degraded states when a public source, AI service or database is offline. Show last-known timestamps and prohibit stale data from silently becoming current.

9. Test strategy and measurable engineering claims

Test Family	Minimum Scenarios	Pass Criterion
Data ingestion	CSV and JSON valid import; malformed data; type/range errors; duplicate IDs; missing columns.	Defined fixtures produce expected issues and records.
Freshness/provenance	Stale record, conflicting source, missing source and timestamp/timezone edge cases.	No unknown/stale value is silently upgraded to verified.
Optimiser	Feasible, infeasible, competing, double-booking and maintenance conflict cases.	Every returned candidate meets model constraints or is rejected.
Independent validator	Mutate a valid run into an invalid allocation and verify detection.	VALIDATED never appears without passing checks.
Dynamic replanning	Resource loss, delay, capacity change, stale source and reset/replay.	Changed inputs and results persisted and comparable.
AI grounding	Ask about known/unknown IDs, invalid plan, stale records and unavailable data.	Cited record evidence; no made-up data; no action authority.
Security/UI	Unauthorised API calls, secret exposure, keyboard flow, small screen, reduced motion.	No high-severity known release blockers; core flow usable.

10. Implementation sequence

Phase	Deliverable	Gate
0 - Inspect	Source/export review; route inventory; existing API and data audit; dependency/build check.	Evidence-based baseline report.
1 - Foundation	Typed domain models, import/validation, clear demo/public labels and canonical source registry.	Import and data-quality tests pass.
2 - Core engine	Generic optimiser, independent validator, immutable run snapshots and run comparison.	Feasible/infeasible cases proved by tests.
3 - Real source/model experiment	One live public-weather connector plus one obtained and documented maintenance dataset.	Real-source metadata and model evaluation report.
4 - Explain and visualise	VAYU tools use actual records; 3D evidence graph shows links to data and runs.	AI/3D integration test and 2D fallback.
5 - Hardening	Auth/rules, accessibility, security tests, failure handling, demo walkthrough and export bundle.	Release checklist, test report and limitations page.

11. Technical conclusion

The design direction is technically coherent if the project prioritises a verifiable core over surface-level feature count. The most original and evaluable contribution is the Evidence-to-Decision Graph connected to provenance-aware data, independent constraint checks and reproducible replanning. The app should not claim to be fully implemented until the actual UI, code, dataset artifacts and tests are inspected.

Project references: [R1] through [R11]; detailed source register is supplied in the separate References Report. The exact public data source coverage, licence and runtime must be revalidated during implementation.